Blockchain Technology- BCSBT603

Module 4- Bitcoin

6th B

Lakshmi B S
Assistant Professor
Dept. of CSE

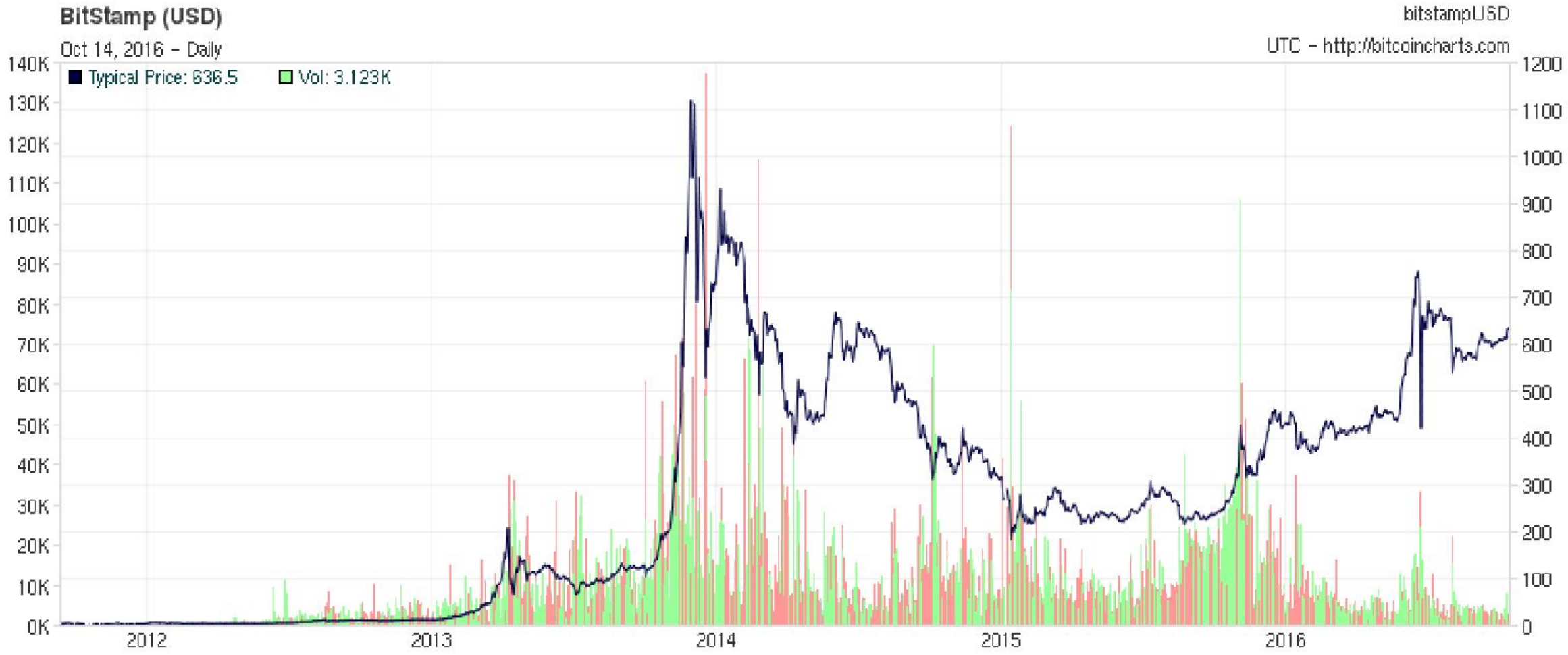
Topics covered...

- Bitcoin definition,
- Transactions -life cycle,
- structure,
- Blockchain: The structure of block
- The structure of a block header,
- Wallets.
- Smart Contracts: History,
- Definition,
- Ricardian contracts

BITCOIN

- Bitcoin is the first application of the blockchain technology.
- Bitcoin has started a revolution with the introduction of the very first fully decentralized digital currency, and one that has proven to be extremely secure and stable.
- In 2008, a paper on bitcoin, *Bitcoin: A Peer-to-Peer Electronic Cash System* was written by *Satoshi Nakamoto*.
- The first key idea introduced in the paper was that purely peer-to-peer electronic cash that does need an intermediary bank to transfer payments between peers.
- Bitcoin is built on decades of cryptographic research such as the research in Merkle trees, hash functions, public key cryptography, and digital signatures.
- Moreover, ideas such as Bit-Gold, b-money, hash-cash, and cryptographic time stamping provided the foundations for bitcoin invention.

- The key issue that has been addressed in bitcoin is an elegant solution to the Byzantine Generals problem along with a practical solution of the double-spend problem.
- The value of bitcoin has increased significantly since 2011, as shown in the following graph.



Bitcoin definition

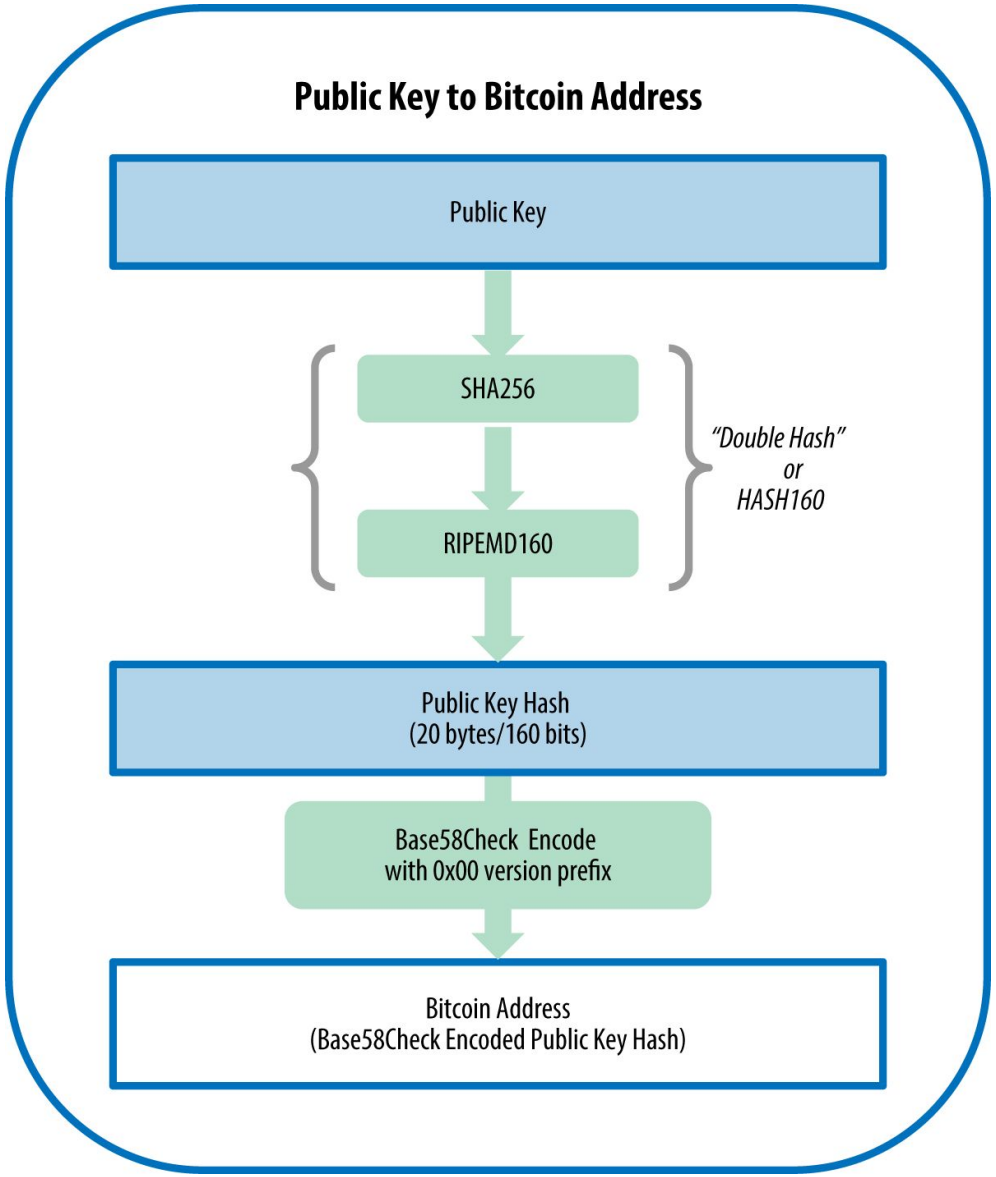
- Bitcoin can be defined in various ways; it's a protocol, a digital currency, and a platform.
- It is a combination of peer-to-peer network, protocols, and software that facilitate the creation and usage of the digital currency named bitcoin.
- Decentralization of currency was made possible for the first time with the invention of bitcoin.
- Moreover, the double spending problem was solved in an elegant and ingenious way in bitcoin.
- Double spending problem arises when, for example, a user sends coins to two different users at the same time and they are verified independently as valid transactions.

Keys and addresses

- Elliptic curve cryptography is used to generate public and private key pairs in the Bitcoin network.
- The bitcoin address is created by taking the corresponding public key of a private key and hashing it twice, first with the SHA256 algorithm and then with RIPEMD160.
- The resultant 160-bit hash is then prefixed with a version number and finally encoded with a Base58Check encoding scheme.
- The bitcoin addresses are 26-35 characters long and begin with digit 1 or 3.
- A typical bitcoin address looks like a string shown here:

1ANAguGG8bikEv2fYsTBnRUmx7QUcK58wt

QR code of a bitcoin address 1ANAguGG8bikEv2fYsTBnRUmx7QUcK58wt


Public Key to Bitcoin Address
Public Key
SHA256
RIPEMD160
"Double Hash"
or
HASH160
Public Key Hash
(20 bytes/160 bits)
Base58Check Encode
with 0x00 version prefix
Bitcoin Address
(Base58Check Encoded Public Key Hash)

- Currently, there are two types of addresses,
- The commonly used P2PKH (Pay To Public Key Hash) and another P2SH(pay to script hash)type, starting with 1 and 3, respectively.
- In the early days, bitcoin used direct Pay-to- Pubkey, which is now superseded by P2PKH.
- However, direct Pay-to-Pubkey is still used in bitcoin for coinbase addresses.
- Addresses should not be used more than once; otherwise, privacy and security issues can arise.

P2PKH Address Example:

1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa

P2SH Address Example:

3J98t1WpEZ73CNmQviecrnyiWrnqRhWNLy

Public keys in bitcoin

- In public key cryptography, public keys are generated from private keys.
- Bitcoin uses ECC (Elliptic curve cryptography)
- A private key is randomly selected and is 256-bit in length.
- Public keys can be presented in an uncompressed or compressed format.
- Public keys are basically x and y coordinates on an elliptic curve and in an uncompressed format and are presented with a prefix of 04 in a hexadecimal format.
- X and Y coordinates are both 32- bit in length

- Keys are identified by various prefixes, described as follows:

- Uncompressed public keys used 0x04 as the prefix
- Compressed public key starts with 0x03 if the y 32-bit part of the public key is odd
- Compressed public key starts with 0x02 if the y 32-bit part of the public key is even

Private keys in bitcoin

- Private keys are basically 256-bit numbers chosen in the range specified by the SECP256K1 ECDSA recommendation.
- Any randomly chosen 256-bit number from 0x1 to 0xFFFF FFFF FFFF FFFF FFFF FFFF FFFF FFFE BAAE DCE6 AF48 A03B BFD2 5E8C D036 4140 is a valid private key.
- Private keys are usually encoded using **Wallet Import Format** (**WIF**) in order to make them easier to copy and use.
- WIF can be converted into private key and vice versa.
- The bitcoin core client also allows the encryption of the wallet that contains the private keys.

Base58Check encoding

- Bitcoin addresses are encoded using the Base58check encoding.
- This encoding is used to limit the confusion between various characters, such as 0 O I l as they can look the same in different fonts.
- The encoding basically takes the binary byte arrays and converts them into human-readable strings.
- This string is composed by utilizing a set of 58 alphanumeric symbols.

Vanity addresses

- As bitcoin addresses are based on base 58 encoding, it is possible to generate addresses that contain human-readable messages. An example is shown as follows:
- **Vanity addresses** in Bitcoin are **customized Bitcoin addresses** that contain a specific pattern or set of characters chosen by the user, usually at the beginning of the address.
- Vanity addresses are generated using a purely brute-force method.
- https://vanitygen.net/
- https://www.mobilefish.com/services/cryptocurrency/cryptocurrency

Public address encoded in QR

Transactions

- Transactions are at the core of the bitcoin ecosystem.
- Transactions can be as simple as just sending some bitcoins to a bitcoin address, or it can be quite complex depending on the requirements.
- Each transaction is composed of at least one input and output.
- Inputs can be thought of as coins being spent that have been created in a previous transaction and outputs as coins being created.
- If a transaction is to sends coins to some other user (a bitcoin address), then it needs to be signed by the sender with their private key and a reference is also required to the previous transaction in order to show the origin of the coins.

The transaction life cycle

1. A user/sender sends a transaction using wallet software or some other interface.
2. The wallet software signs the transaction using the sender's private key.
3. The transaction is broadcasted to the Bitcoin network using a flooding algorithm.
4. Mining nodes include this transaction in the next block to be mined.
5. Mining starts once a miner who solves the Proof of Work problem broadcasts the newly mined block to the network.
6. The nodes verify the block and propagate the block further, and confirmation starts to generate.
7. Finally, the confirmations start to appear in the receiver's wallet and after approximately six confirmations, the transaction is considered finalized and confirmed.

The transaction structure

- A transaction at a high level contains metadata, inputs, and outputs. Transactions are combined to create a block. The transaction structure is shown in the following table:

Field	Size	Description
Version Number	4 bytes	Used to specify rules to be used by the miners and nodes for transaction processing.
Input counter	1 bytes – 9 bytes	The number of inputs included in the transaction.
list of inputs	variable	Each input is composed of several fields, including Previous transaction hash, Previous Txout-index, Txin-script length, Txin-script, and optional sequence number. The first transaction in a block is also called a coinbase transaction. It specifies one or more transaction inputs.
Out-counter	1 bytes – 9 bytes	A positive integer representing the number of outputs.
list of outputs	variable	Outputs included in the transaction.
lock_time	4 bytes	This defines the earliest time when a transaction becomes valid. It is either a Unix timestamp or a block number.

- **MetaData**: This part of the transaction contains some values such as the size of the transaction, the number of inputs and outputs, the hash of the transaction, and a lock_time field. Every transaction has a prefix specifying the version number.
- **Inputs**: Generally, each input spends a previous output. Each output is considered an **Unspent Transaction Output** (**UTXO**) until an input consumes it.
- **Outputs**: Outputs have only two fields, and they contain instructions for the sending of bitcoins. The first field contains the amount of Satoshis, whereas the second field is a locking script that contains the conditions that need to be met in order for the output to be spent.
- **Verification:** Verification is performed using bitcoin's scripting language.

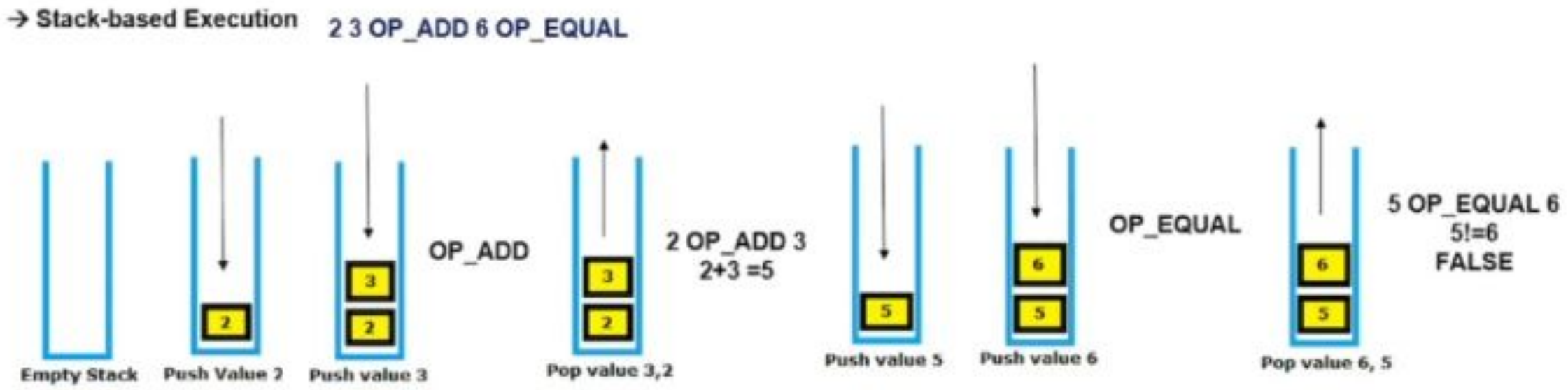
A sample transaction is shown as follows:


```
{
  "txid": "08af7960ca9255c67686296fb65452ed3f96f18831c9a3d8ea552e4ccee5c4af",
  "hash": "08af7960ca9255c67686296fb65452ed3f96f18831c9a3d8ea552e4ccee5c4af",
  "size": 226,
  "vsize": 226,
  "version": 1,
  "locktime": 969523,
  "vin": [
    {
 "txid": "3e553260a0a94860f7043eb6576e15e6cfeb2990aea961210ae1fde328bb08b0",
 "vout": 1,
 "scriptSig": {
 "asm":
"3045022100cfb31edabc62c82b41d12f651d2e3e013ee1a7ee2bb4526f3dda640e6d8d224502207d8d1d8e41350b9cdf36f389f942ab68c12
f113fe99014f5d6df6610407877d2[ALL] 037bc82d0078993f6943e7ff6e82e82da600f34edc8bca136331a9901c8bb60b0d",
 "hex":
"483045022100cfb31edabc62c82b41d12f651d2e3e013ee1a7ee2bb4526f3dda640e6d8d224502207d8d1d8e41350b9cdf36f389f942ab68c
12f113fe99014f5d6df6610407877d20121037bc82d0078993f6943e7ff6e82e82da600f34edc8bca136331a9901c8bb60b0d"
      },
      "sequence": 4294967294
    }
  ],
  "vout": [
    {
      "value": 2.30000000,
      "n": 0,
      "scriptPubKey": {
        "asm": "OP_DUP OP_HASH160 07e78644a61343068fa8d4940a79976e758ac6ef OP_EQUALVERIFY OP_CHECKSIG",
        "hex": "76a91407e78644a61343068fa8d4940a79976e758ac6ef88ac",
        "reqSigs": 1,
        "type": "pubkeyhash",
        "addresses": [
          "mgEkNzxV3qbYtDKEKTvo1VpgJ63Au619q2"
}}}
```

The script language

- Bitcoin uses a simple stack-based language called *script* to describe how bitcoins can be spent and transferred.
- This scripting language is based on a Forth-like syntax and uses a reverse polish notation in which every operand is followed by its operators.
- It is evaluated from the left to the right using a **Last in First Out** (**LIFO**) stack.
- Scripts use various Opcodes or instructions to define their operation.
- Opcodes are also known as words, commands, or functions. Earlier versions of the bitcoin node had a few Opcodes that are no longer used due to bugs discovered in their design.
- The various categories of the scripting Opcodes are constants, flow control, stack, bitwise logic, splice, and arithmetic, cryptography, and lock time.

Example


→ Stack-based Execution
2 3 OP_ADD 6 OP_EQUAL
Empty Stack
Push Value 2
Push value 3
OP_ADD
Pop value 3,2
2 OP_ADD 3
2+3 =5
Push value 5
Push value 6
OP_EQUAL
Pop value 6, 5
5 OP_EQUAL 6
5!=6
FALSE

/**Script opcodes*/

```
enum opcodetype
{
    // push value
    OP_0 = 0x00,
    OP_FALSE = OP_0,
    OP_PUSHDATA1 = 0x4c,
    OP_PUSHDATA2 = 0x4d,
    OP_PUSHDATA4 = 0x4e,
    OP_1NEGATE = 0x4f,
    OP_RESERVED = 0x50,
    OP_1 = 0x51,
    OP_TRUE=OP_1,
    OP_2 = 0x52,
    OP_3 = 0x53,
    OP_4 = 0x54,
    OP_5 = 0x55,
    OP_6 = 0x56,
    OP_7 = 0x57,
    OP_8 = 0x58,
    OP_9 = 0x59,
    OP_10 = 0x5a,
    OP_11 = 0x5b,
    OP_12 = 0x5c,
    OP_13 = 0x5d,
    OP_14 = 0x5e,
    OP_15 = 0x5f,
    OP_16 = 0x60,
```

//control

```
OP_NOP = 0x61,
OP_VER = 0x62,
OP_IF = 0x63,
OP_NOTIF = 0x64,
OP_VERIF = 0x65,
OP_VERNOTIF = 0x66,
OP_ELSE = 0x67,
OP_ENDIF = 0x68,
OP_VERIFY = 0x69,
OP_RETURN = 0x6a,

// stack ops
OP_TOALTSTACK = 0x6b,
OP_FROMALTSTACK = 0x6c,
OP_2DROP = 0x6d,
OP_2DUP = 0x6e,
OP_3DUP = 0x6f,
OP_2OVER = 0x70,
OP_2ROT = 0x71,
OP_2SWAP = 0x72,
OP_IFDUP = 0x73,
OP_DEPTH = 0x74,
OP_DROP = 0x75,
OP_DUP = 0x76,
OP_NIP = 0x77,
OP_OVER = 0x78,
OP_PICK = 0x79,
OP_ROLL = 0x7a,
OP_ROT = 0x7b,
OP_SWAP = 0x7c,
OP_TUCK = 0x7d,
```

//splice ops

```
OP_CAT = 0x7e,
OP_SUBSTR = 0x7f,
OP_LEFT = 0x80,
OP_RIGHT = 0x81,
OP_SIZE = 0x82,

// bit logic
OP_INVERT = 0x83,
OP_AND = 0x84,
OP_OR = 0x85,
OP_XOR = 0x86,
OP_EQUAL = 0x87,
OP_EQUALVERIFY = 0x88,
OP_RESERVED1 = 0x89,
OP_RESERVED2 = 0x8a,

// numeric
OP_1ADD = 0x8b,
OP_1SUB = 0x8c,
OP_2MUL = 0x8d,
OP_2DIV = 0x8e,
OP_NEGATE = 0x8f,
OP_ABS = 0x90,
OP_NOT = 0x91,
OP_0NOTEQUAL = 0x92,
```

//OP ADD=0x93

```
OP_SUB = 0x94,
OP_MUL = 0x95,
OP_DIV = 0x96,
OP_MOD = 0x97,
OP_LSHIFT = 0x98,
OP_RSHIFT = 0x99,

OP_BOOLAND = 0x9a,
OP_BOOLOR = 0x9b,
OP_NUMEQUAL = 0x9c,
OP_NUMEQUALVERIFY = 0x9d,
OP_NUMNOTEQUAL = 0x9e,
OP_LESSTHAN = 0x9f,
OP_GREATERTHAN = 0xa0,
OP_LESSTHANOREQUAL = 0xa1,
OP_GREATERTHANOREQUAL = 0xa2,
OP_MIN = 0xa3,
OP_MAX = 0xa4,

OP_WITHIN = 0xa5,
```

//crypto

```
OP_RIPEMD160 = 0xa6,
OP_SHA1 = 0xa7,
OP_SHA256 = 0xa8,
OP_HASH160 = 0xa9,
OP_HASH256 = 0xaa,
OP_CODESEPARATOR = 0xab,
OP_CHECKSIG = 0xac,
OP_CHECKSIGVERIFY = 0xad,
OP_CHECKMULTISIG = 0xae,
OP_CHECKMULTISIGVERIFY = 0xaf,

// expansion
OP_NOP1 = 0xb0,
OP_CHECKLOCKTIMEVERIFY = 0xb1,
OP_NOP2 = OP_CHECKLOCKTIMEVERIFY,
OP_CHECKSEQUENCEVERIFY = 0xb2,
OP_NOP3 = OP_CHECKSEQUENCEVERIFY,
OP_NOP4 = 0xb3,
OP_NOP5 = 0xb4,
OP_NOP6 = 0xb5,
OP_NOP7 = 0xb6,
OP_NOP8 = 0xb7,
OP_NOP9 = 0xb8,
OP_NOP10 = 0xb9,

OP_INVALIDOPCODE = 0xff,
```

OP_CODES IN BITCOIN SCRIPT

Commonly used Opcodes

- All Opcodes are declared in the script.h file in the bitcoin reference client source code.

Opcode	Description
OP_CHECKSIG	This takes a public key and signature and validates the signature of the hash of the transaction. If it matches, then TRUE is pushed onto the stack; otherwise, FALSE is pushed.
OP_EQUAL	This returns 1 if the inputs are exactly equal; otherwise, 0 is returned.
OP_DUP	This duplicates the top item in the stack.
OP_HASH160	The input is hashed twice, first with SHA-256 and then with RIPEMD-160.
OP_VERIFY	This marks the transaction as invalid if the top stack value is not true.
OP_EQUALVERIFY	This is the same as OP_EQUAL, but it runs OP_VERIFY afterwards.
OP_CHECKMULTISIG	This takes the first signature and compares it against each public key until a match is found and repeats this process until all signatures are checked. If all signatures turn out to be valid, then a value of 1 is returned as a result; otherwise, 0 is returned.

Types of transaction

- **Pay to Public Key Hash** (**P2PKH**): P2PKH is the most commonly used transaction type and is used to send transactions to the bitcoin addresses. The format of the transaction is shown as follows:

```
ScriptPubKey: OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY
OP_CHECKSIG

ScriptSig: <sig> <pubKey>
```

- **Pay to Script Hash** (**P2SH**): P2SH is used in order to send transactions to a script hash (that is, the addresses starting with 3) and was standardized in BIP16. In addition to passing the script, the redeem script is also evaluated and must be valid. The template is shown as follows:

```
ScriptPubKey: OP_HASH160 <redeemScriptHash> OP_EQUAL

ScriptSig: [<sig>...<sign>] <redeemScript>
```

- **MultiSig (Pay to MultiSig):** M of n multisignature transaction script is a complex type of script where it is possible to construct a script that required multiple signatures to be valid in order to redeem a transaction. Various complex transactions such as escrow and deposits can be built using this script. The template is shown here:

```
ScriptPubKey: <m> <pubKey> [<pubKey> . . . ] <n> OP_CHECKMULTISIG

ScriptSig: 0 [<sig > . . . <sign>]
```

- **Pay to Pubkey:** This script is a very simple script that is commonly used in coinbase transactions. It is now obsolete and was used in an old version of bitcoin. The public key is stored within the script in this case, and the unlocking script is required to sign the transaction with the private key.

The template is shown as follows:

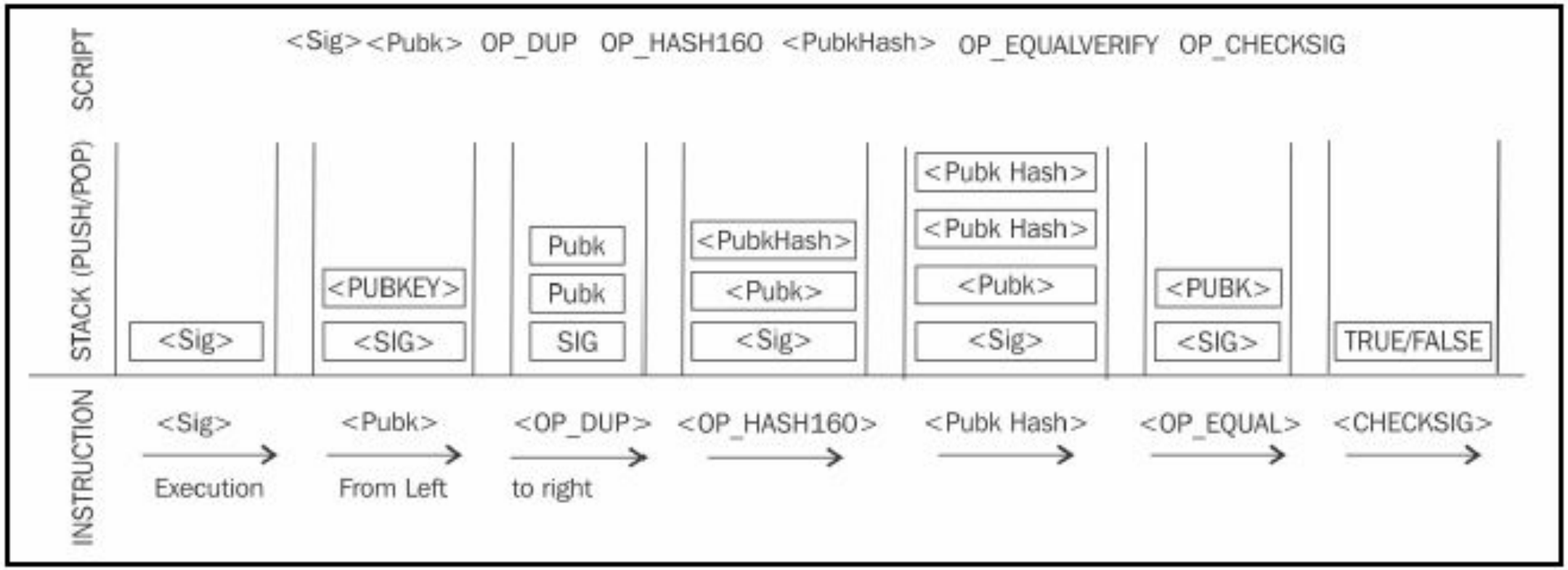
<PubKey> OP_CHECKSIG

- **Null data/OP_RETURN:** This script is used to store arbitrary data on the blockchain for a fee. The limit of the message is 40 bytes. The output of this script is unredeemable because OP_RETURN will fail the validation in any case. ScriptSig is not required in this case.

The template is very simple and is shown as follows:

OP_RETURN <data>

A P2PKH script execution is shown as follows:



P2PKH script execution

Blockchain

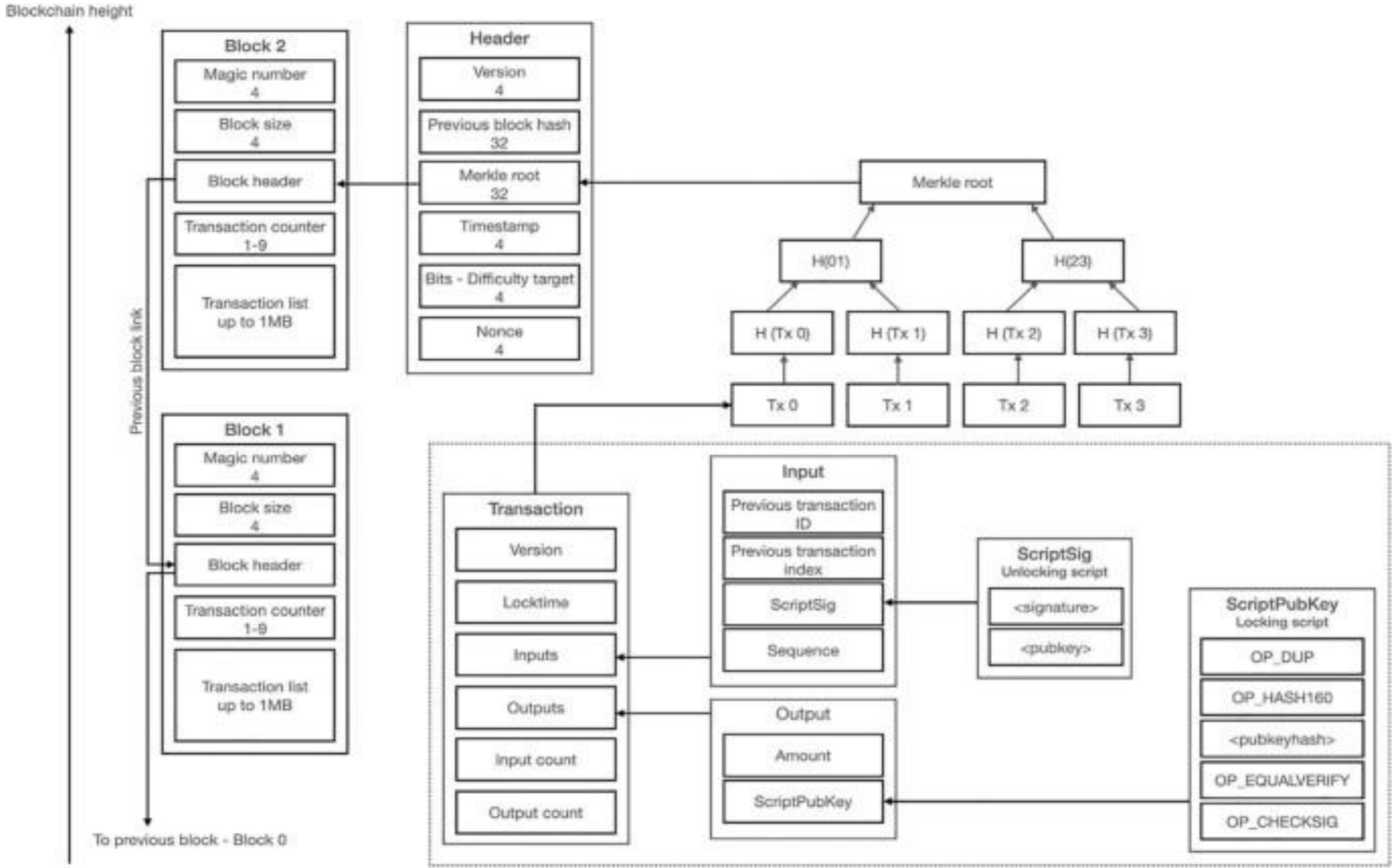
- Blockchain is a public ledger of a timestamped, ordered, and immutable list of all transactions on the bitcoin network.
- Each block is identified by a hash in the chain and is linked to its previous block by referencing the previous block's hash.
- Blockchain is a distributed ledger of transaction.

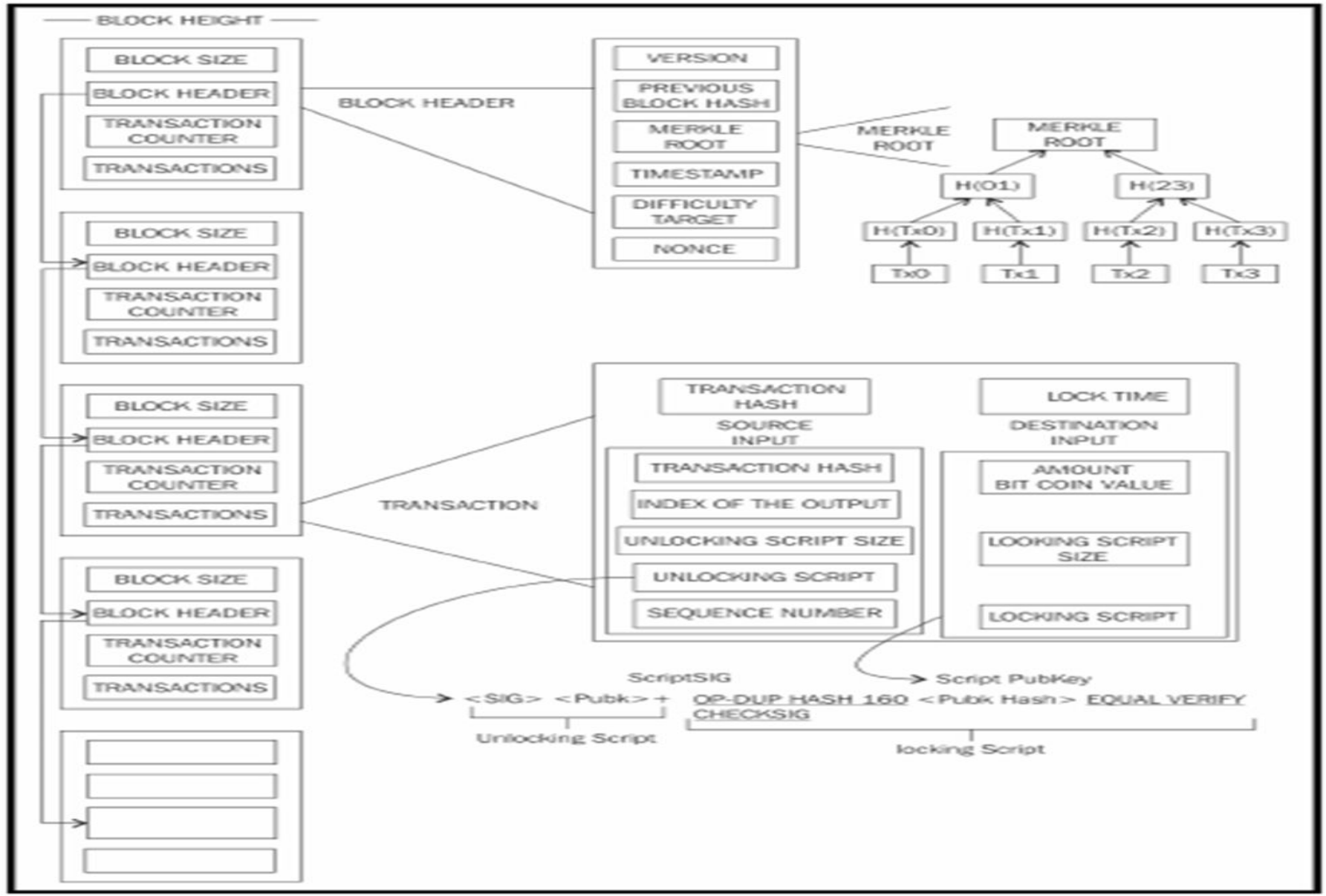
Data structure of a Bitcoin block

Field	Size	Description
Block size	4 bytes	The size of the block
Block header	80 bytes	This includes fields from the block header described in the next section.
Transaction counter	Variable	The field contains the total number of transactions in the block. Size ranges from 1-9 bytes.
Transactions	Variable	All transactions in the block.

Data structure of block header

Field	Size	Description
Version	4 bytes	The block version number that dictates the block validation rules follow.
Previous block header hash	32 bytes	This is a double SHA256 hash of the previous blocks header.
Merkle root hash	32 bytes	This is a double SHA256 hash of the merkle tree of all transactions included in the block.
Timestamp	4 bytes	Contains the approximate creation time of the block in Unix-epoch time format.
Difficulty target	4 bytes	Current difficulty target of the network
Nonce	4 bytes	Arbitrary number that miners change repeatedly to produce a hash that is lower than the difficulty target.

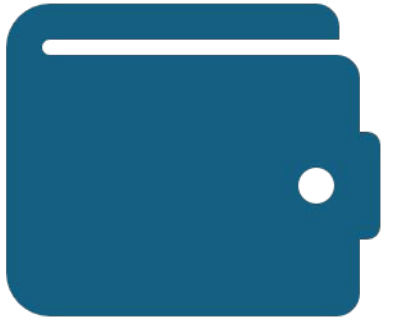

Blockchain height
Block 2
Magic number
4
Block size
4
Block header
Transaction counter
1-9
Transaction list
up to 1MB
Header
Version
4
Previous block hash
32
Merkle root
32
Timestamp
4
Bits - Difficulty target
4
Nonce
4
Merkle root
H(01)
H(23)
H (Tx 0)
H (Tx 1)
H (Tx 2)
H (Tx 3)
Tx 0
Tx 1
Tx 2
Tx 3
Previous block link
Block 1
Magic number
4
Block size
4
Block header
Transaction counter
1-9
Transaction list
up to 1MB
To previous block - Block 0
Transaction
Version
Locktime
Inputs
Outputs
Input count
Output count
Input
Previous transaction
ID
Previous transaction
index
ScriptSig
Sequence
ScriptSig
Unlocking script
<signature>
<pubkey>
Output
Amount
ScriptPubKey
ScriptPubKey
Locking script
OP_DUP
OP_HASH160
<pubkeyhash>
OP_EQUALVERIFY
OP_CHECKSIG



A visualization of blockchain, block, block header, transaction and script

Wallets

- The wallet software is used to store private or public keys and bitcoin address.
- It performs various functions, such as receiving and sending bitcoins.
- Private keys can be generated in different ways and are used by different types of wallets.
- Wallets do not store any coins, and there is no concept of wallets storing balance or coins for a user.
- In the bitcoin network, *coins* do not exist; instead, only transaction information is stored on the blockchain

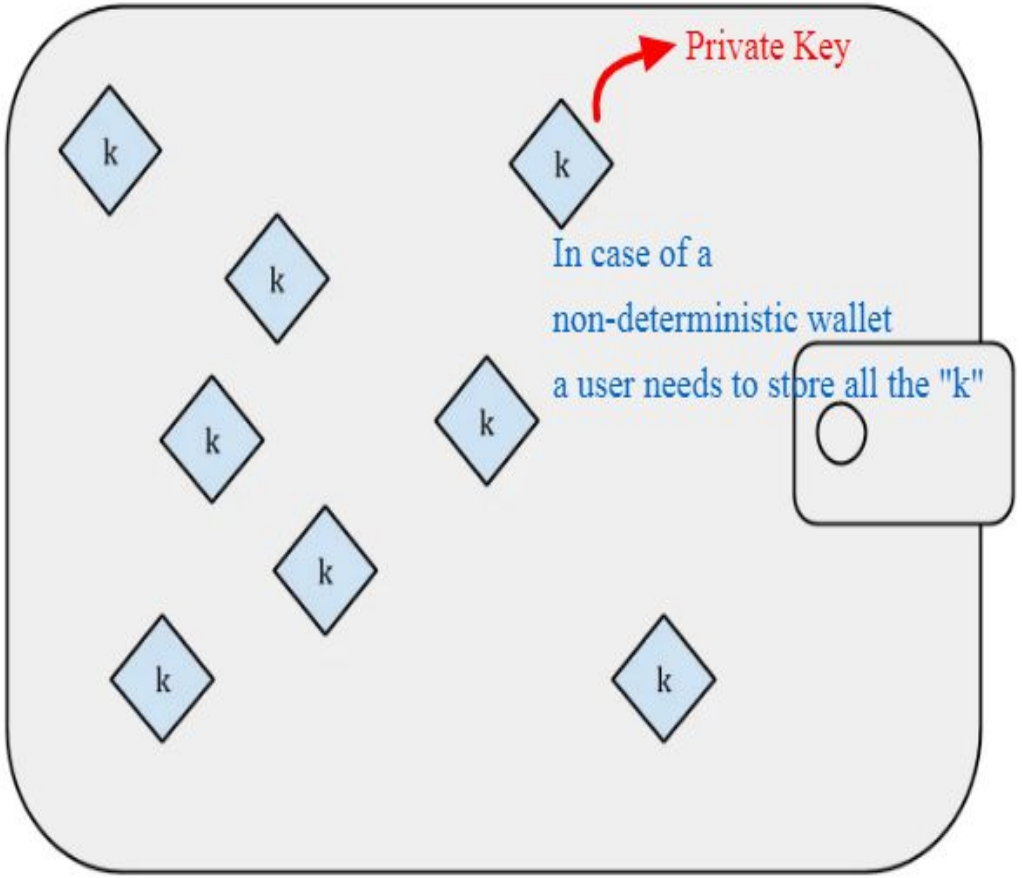


Wallet types

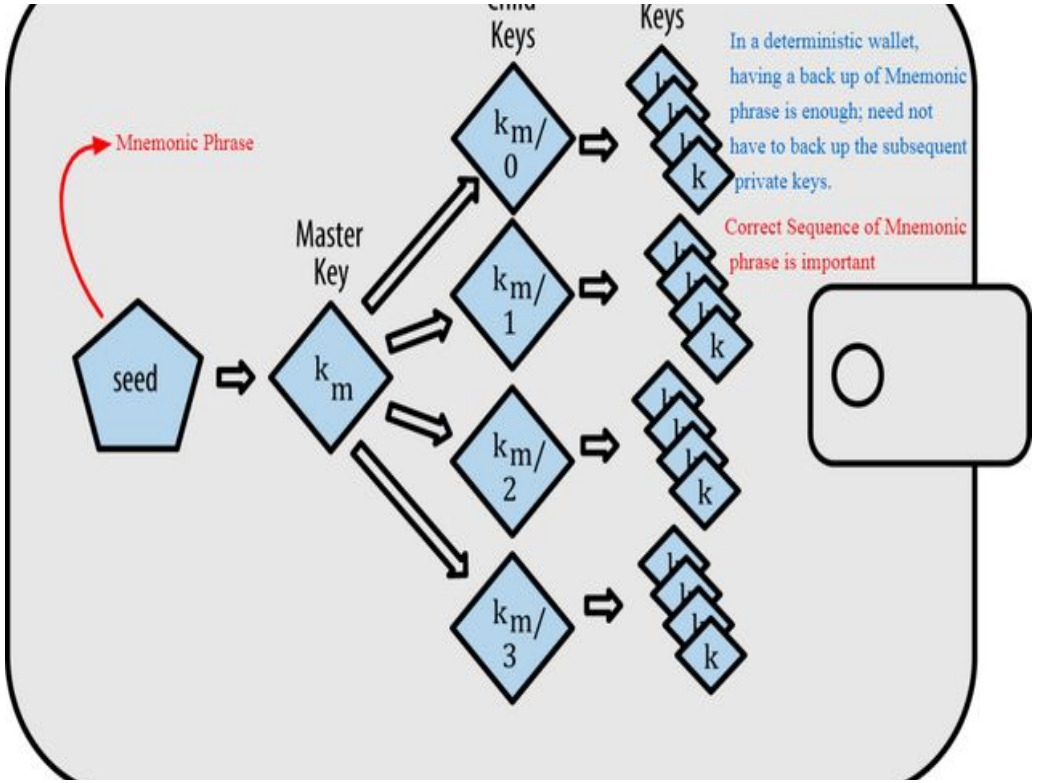
- Different types of wallets that are used to store private keys.
- **Non-deterministic wallets**
- **Deterministic wallets**
- **Hierarchical deterministic wallets**
- **Brain wallets**
- **Paper wallets**
- **Hardware wallets**
- **Online wallets**
- **Mobile wallets**

Non-deterministic wallets

- These wallets contain **randomly** generated private keys and are also called *Just a Bunch of Key wallets*.
- The bitcoin **core client** generates some keys when first started and generates keys as and when required.
- Managing a large number of keys is very **difficult**.
- There is a need to create regular **backups** of the keys and protect them appropriately in order to prevent theft or loss.



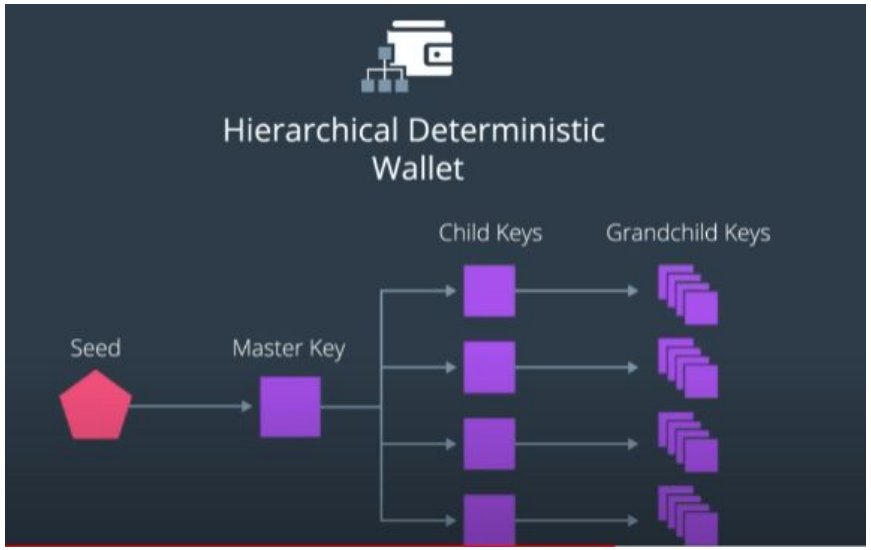
Deterministic wallets



- Keys are derived out of a **seed** value usually of 12 or 24 words via hash functions.
- This seed number is generated **randomly** and is commonly represented by human-readable *mnemonic code* words.
- Mnemonic code words are defined in **BIP39**.
- Private key management comparatively **easier.**

Hierarchical deterministic wallets

- Defined in BIP32 and BIP44, HD wallets store keys in a **tree structure** derived from a seed.
- The seed generates the parent key (master key), which is used to generate child keys and, subsequently, grandchild keys.
- The complete hierarchy of private keys in an HD wallet is easily **recoverable** if the master private key is known.
- It is because of this property that HD wallets are very easy to **maintain** and are highly **portable**.



Brain wallets

- The master private key can also be derived from the hash of passwords that are memorized.
- The key idea is that this passphrase is used to derive the private key and if used in HD wallets, this can result in a full HD wallet that is derived from a single memorized password.
- This method is prone to password guessing and brute force attacks but techniques such as **key stretching** can be used to slow down the progress made by the attacker.
- Example : BitAddress.org

Paper wallets

- This is a paper-based wallet with the required key material printed on it and It requires physical security to be stored.
- Paper wallets can be generated online from various service providers, such as **https://bitcoinpaperwallet.com/ or https://www.bitaddress.org/.**

Hardware wallets

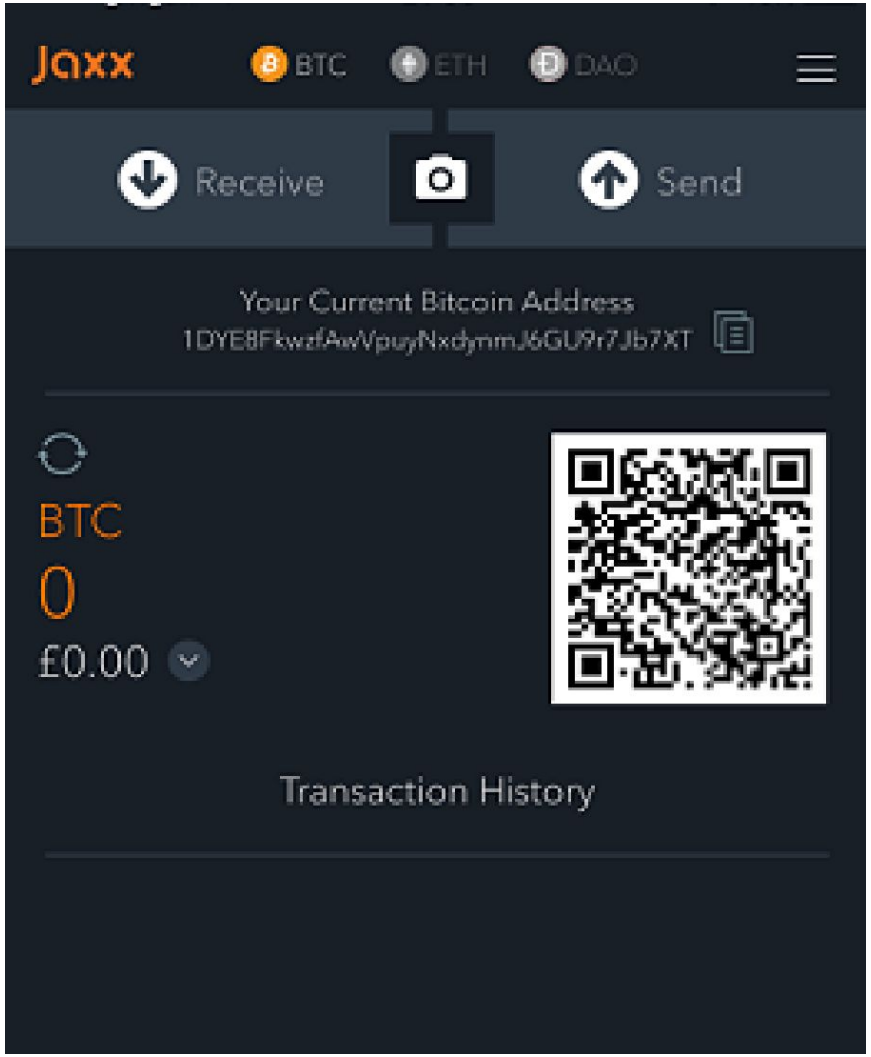
- Another method is to use a tamper-resistant device to store keys.
- This tamper-resistant device can be custom-built or with the advent of **NFC**-enabled phones, this can also be a secure element (SE) in NFC phones.
- **Trezor** and **Ledger** wallets (various types) are the most commonly used bitcoin hardware wallets.

Online wallets

- They are stored entirely in online and are provided as a service usually via cloud.
- They provide a web interface to the users to manage their wallets and perform various functions such as making and receiving payments.
- Easy to use but imply that the user trust the online wallet service provider.
- Example : Coinbase.com

Mobile wallets

- They are installed on mobile devices. They can provide various methods to make payments.
- At most the notable ability to use smart phone cameras to scan QR codes quickly and make payments.
- Mobile wallets are available for the Android platform and iOS, for example, breadwallet, copay, and Jaxx.



Smart Contracts

- Smart contracts are described by *Szabo* as follows:

"A smart contract is a computerized transaction protocol that executes the terms of a contract. The general objectives are to satisfy common contractual conditions (such as payment terms, liens, confidentiality, and even enforcement), minimize exceptions both malicious and accidental, and minimize the need for trusted intermediaries. Related economic goals include lowering fraud loss, arbitrations and enforcement costs, and other transaction costs."

Definition

A smart contract is a secure and unstoppable computer program representing an agreement that is automatically executable and enforceable.

Smart Contracts

- Smart contracts usually operate by managing their internal state using a state machine model.
- This allows development of an effective framework for programming smart contracts, where the state of a contract is advanced further based on some predefined criteria and conditions.
- **There is also on-going debate on the question of whether code is acceptable as a contract in a court of law.**
- This raises several questions around how a smart contract can be legally binding:

 can it be developed in such a way that it is readily acceptable and understandable in a court of law?

 How can dispute resolution be implemented within the code, and is it possible?

- The preceding questions open up various possibilities, such as making smart contract code readable not only by machines but also by people. If humans and machines can both understand the code written in a smart contract it might be more acceptable in legal situations, as opposed to just a piece of code that no-one understands except for programmers.

- Smart contracts are inherently required to be deterministic in nature.
- This property will allow a smart contract to be run by any node on a network and achieve the same result. If the result differs even slightly between nodes, consensus then cannot be achieved.
- A smart contract has the following four properties:

 Automatically executable

 Enforceable

 Semantically sound

 Secure and unstoppable.

- This issue of interpretation was addressed by Ian Grigg with his invention of **Ricardian contracts,**

Ricardian contracts

- Ricardian contracts were originally proposed in the *Financial Cryptography in 7 Layers* paper by *Ian Grigg* in late 1990s.
- These contracts were used initially in a bond trading and payment system called **Ricardo**.
- The key idea is to write a document which is understandable and acceptable by both a court of law and computer software.
- Ricardian contracts address the challenge of issuance of value over the Internet.
- It identifies the issuer and captures all the terms and clauses of the contract in a document in order to make it acceptable as a legally binding contract.

Example:

- Let’s say Alice wants to purchase digital art from Bob.

1. Human-Readable Legal Text:

“This agreement is between Alice (buyer) and Bob (seller) for the purchase of a digital artwork titled “Sunset Dreams.” Alice agrees to pay 1 ETH to Bob. Upon receipt of payment, Bob will transfer ownership of the NFT (token ID: 12345) to Alice. Both parties agree to the terms as outlined here.”

2. Machine-Readable Data (Embedded or Attached):

```
{
  "buyer": "0xAliceAddress",
  "seller": "0xBobAddress",
  "price": "1 ETH",
  "nft_token_id": 12345,
  "nft_contract": "0xNFTContractAddress",
  "conditions": {
    "transfer_upon_payment": true
  },
  "hash": "0xa3f5c...e91a2"  // Hash of the full human-readable contract
}
```

3. Digitally Signed by Both Parties:

Both Alice and Bob cryptographically sign the contract with their private keys, locking in agreement.

- A Ricardian contract is a document that has several of the following properties:

 A contract offered by an issuer to holders

 A valuable right held by holders, and managed by the issuer

 Easily readable by people (like a contract on paper)

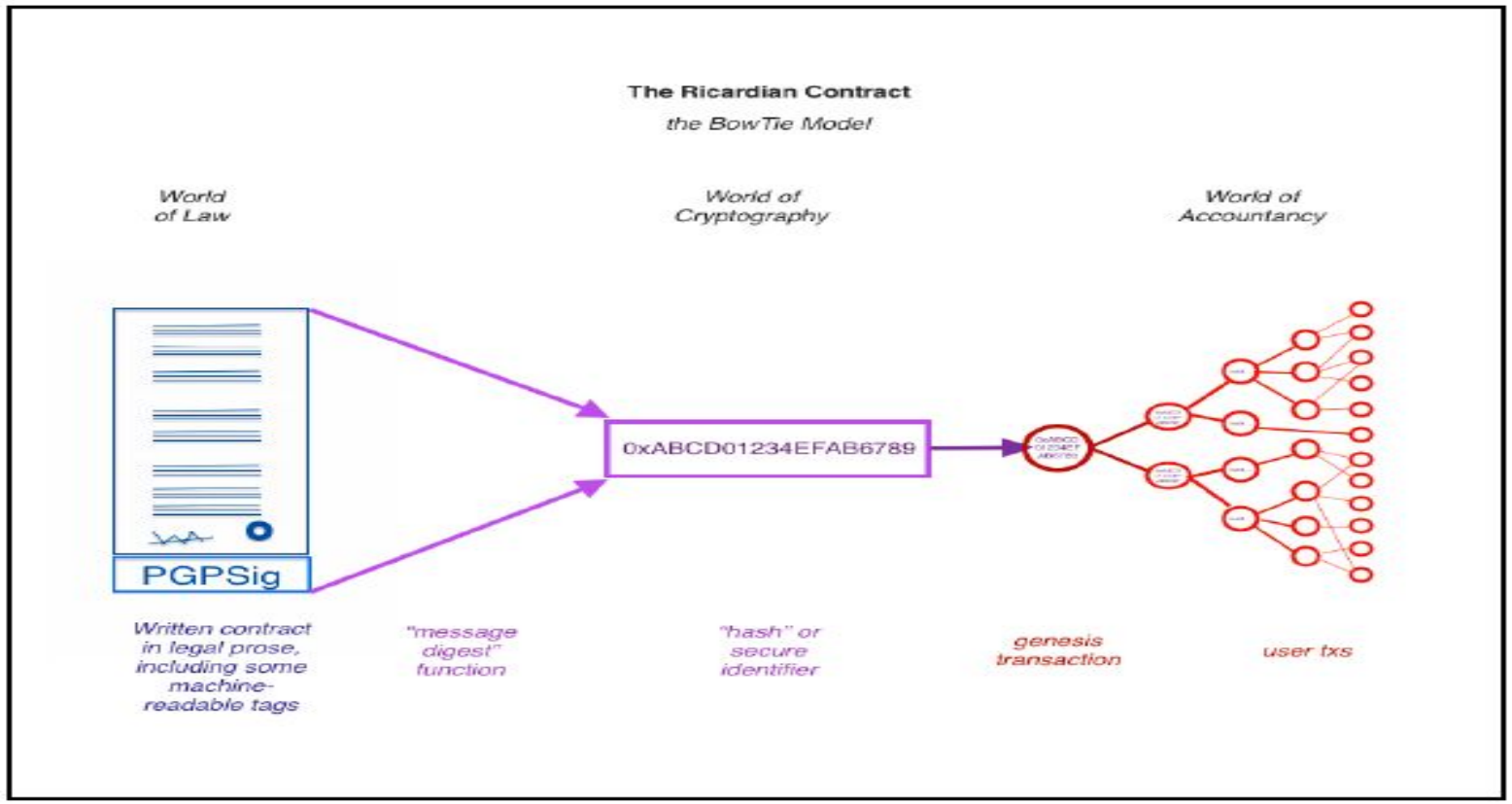
 Readable by programs (parseable, like a database)

 Digitally signed

 Carries the keys and server information

 Allied with a unique and secure identifier

- The contracts are implemented by producing a single document that contains the terms of the contract in legal prose and the required machine-readable tags.
- This document is digitally signed by the issuer using their private key.
- This document is then hashed using a message digest function to produce a hash by which the document can be identified.
- This hash is then further used and signed by parties during the performance of the contract in order to link each transaction, with the identifier hash thus serving as evidence of intent.



Ricardian contracts, bowtie diagram

World of Law

- Represents the **human-readable** legal prose.
- This is a **written contract** (in natural language) that may include **machine-readable tags** (like metadata).
- It is **digitally signed** to ensure authenticity.
- This written document is then run through a **message digest function** (like SHA-256) to create a **unique hash**.

World of Cryptography

- The hash acts as a secured identifiers.
- The identifiers becomes the link between legal document and digital signature.

World of Accountancy:

- The identifier (hash) is used in the **genesis transaction**, which initiates activity on a blockchain or ledger.
- From this transaction, a whole **network of user transactions** branches out.
- These transactions are **programmatically tied back** to the original legal agreement via the secure identifier.

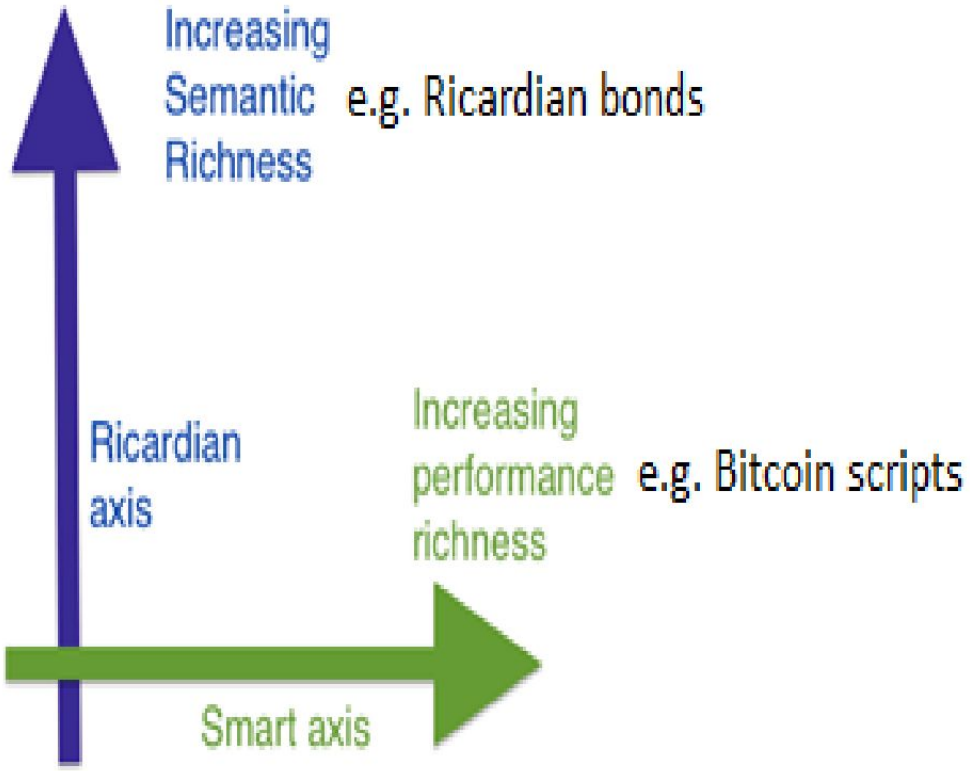
Ricardian Contracts

- A Ricardian contract is different from a smart contract in the sense that a smart contract does not include any contractual document and is focused purely on the execution of the contract.
- A Ricardian contract, on the other hand, is more concerned with the semantic richness and production of a document that contains contractual legal prose.
- The semantics of a contract can be divided into two types:

operational semantics and denotational semantics.

- Operational Semantics : defines the actual execution, correctness and safety of the contract.
- Denotational Semantics: Encompasses both the denotational and operational semantics of a legal agreement. Also capable of encoding legal prose and code (business logic) in it

- At bitcoin, a very simple implementation of a smart contract can be observed which is fully oriented towards the execution of the contract, whereas a Ricardian contract is more geared towards producing a document that is understandable by humans, with some parts that a computer program can understand.
- This can be viewed as legal semantics vs operational performance (semantics vs performance) as shown in the following diagram.



- A Ricardian contract can be represented as a tuple of three objects, namely *Prose*, *parameters* and *code*.
- Prose represents the legal contract in regular language;
- code represents the program that is a computer-understandable representation of legal prose; and
- parameters join the appropriate parts of the legal contract to the equivalent code.
- Ricardian contracts have been implemented in many systems, such as CommonAccord, OpenBazaar, OpenAssets, and Askemos.

Example: Ricardian Contract for a Token Sale

RicardianContract = (Prose, Parameters, Code)

1. **Prose** (Human-readable legal text):

“This Agreement, made on April 20, 2025, is between Alice (the "Seller") and Bob (the "Buyer"). The Seller agrees to sell, and the Buyer agrees to purchase, 1,000 utility tokens (TOK) at a price of 0.05 ETH per token. Payment shall be made via Ethereum, and tokens will be transferred to the Buyer's wallet upon confirmation of payment. This contract is binding and enforceable under applicable law”

2. **Parameters** (Structured data for both human and machine)


```
{
  "seller": "0xABC123...",
  "buyer": "0xDEF456...",
  "token_symbol": "TOK",
  "token_amount": 1000,
  "price_per_token_eth": 0.05,
  "payment_currency": "ETH",
  "contract_date": "2025-04-20"
}
```

3. **Code** (Smart contract or executable logic)

```
// Simplified ERC20 token sale logic
function buyTokens() public payable {
    require(msg.value == tokenAmount * pricePerToken);
    token.transfer(msg.sender, tokenAmount);
}
```

Smart contract templates

- Smart contracts can be implemented for any industry where required but most current use cases are related to the financial industry.
- Recent work in smart contract specific to the financial industry has proposed the idea of smart contract templates.
- The idea is to build standard templates that provide a framework to support legal agreements for financial instruments.
- This was proposed by *Clack et al* in their paper named *Smart Contract Templates: Foundations, design landscape and research directions*.

- The paper also proposed that domain specific languages should be built in order to support design and implementation of smart contract templates.
- A language named *CLACK*, a common language for augmented contract knowledge has been proposed and research has begun to develop the language.
- This language is intended to be very rich and provide a large variety of functions ranging from supporting legal prose to the ability to be executed on multiple platforms and cryptographic functions.

Smart contracts run on blockchains, which are **deterministic and isolated environments**. This ensures security and consistency — but it also means that **smart contracts cannot natively access off-chain (external) data**, such as:

- Asset prices (e.g., ETH/USD)
- Weather data
- Sports scores
- IoT sensor readings
- KYC/AML information
- Legal rulings or news events
- Yet, many business use cases — especially in **finance**, **insurance**, **supply chain**, etc. — depend on **real-world data**.

What is an Oracle?

- An **oracle** is a trusted third-party service that **fetches, verifies, and feeds external data** into the blockchain so that smart contracts can use it.

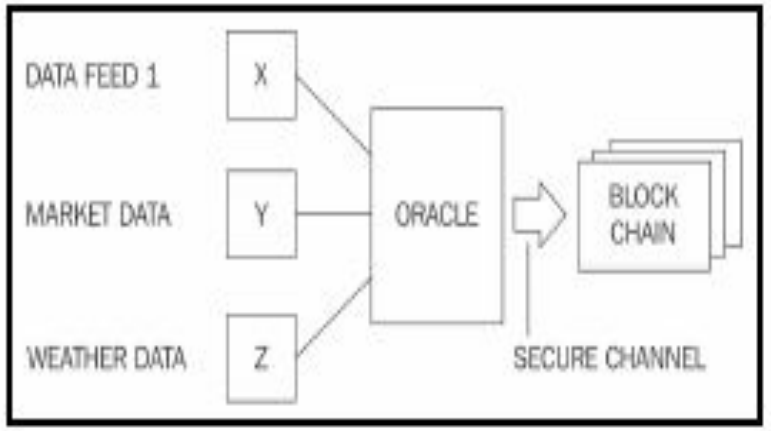
Oracles

- Oracles are an important component of the smart contract ecosystem.
- Oracles can be used to provide external data to smart contracts.
- Depending on the industry and requirements, Oracles can deliver different types of data ranging from weather reports, real-world news, and corporate actions to data coming from **Internet of Things** (**IoT**) devices.
- Oracles are also capable of digitally signing the data proving that the source of the data is authentic.
- Smart contracts can then subscribe to the Oracles, and the smart contracts can either pull the data, or Oracles can push the data to the smart contracts.

- Oracles should not be able to manipulate the data they provide and must be able to provide authentic data.
- Even though Oracles are trusted, it may still be possible in some cases that the data is incorrect due to manipulation.
- Therefore, it is necessary that Oracles are unable to change the data.
- It might be acceptable for a smart contract designer to accept data for an oracle that is provided by a large reputable trusted third party, but the issue of centralization still remains. These types of Oracles can be called standard or simple Oracles.
- Another type of Oracle, which essentially emerged due to the decentralization requirements, can be called decentralized Oracles.
- These types of Oracles can be built based on some distributed mechanism.
- It can also be envisaged that the Oracles can themselves source data from another blockchain which is driven by distributed consensus, thus ensuring the authenticity of data.
- For example, one institution running their own private blockchain can publish their data feed via an Oracle that can then be consumed by other blockchains.

- Another concept of hardware Oracles is also introduced by researchers where real-world data from physical devices is required.
- For example, this can be used in telemetry and IoT. However, this approach however requires a mechanism in which hardware devices cannot be tampered with.
- This can be achieved by using tamper-proof devices.

The following diagram shows a generic model of an oracle and smart contract ecosystem:



A simplified model of an oracle interacting with smart contract on blockchain

Deploying smart contracts on a blockchain

- Smart contracts deploy them on a blockchain due to the distributed consensus mechanism provided by blockchain.
- Ethereum is an example of a blockchain that natively supports the development and deployment of smart contracts.
- Smart contracts on Ethereum blockchain are usually part of a larger application such as **Decentralized Autonomous organization** (**DAOs**).
- In bitcoin blockchain the lock_time field in the bitcoin transaction can be seen as an enabler of a basic version of a smart contract.
- The lock_time field enables a transaction to be locked until a specified time or after a number of blocks, thus enforcing a basic contract that a certain transaction can only be unlocked if certain conditions (elapsed time or number of blocks) is met.
- However, this is very limited in nature and should be only viewed as an example of a basic smart contract